第 19 章：LLM 原理基础

版本: v2.0（润色版）
作者: 内容撰写专家（LLM 原理 + Prompt 工程方向）
状态: 润色完成
最后更新: 2026-03-23

【本章导读】

理解 LLM 的工作原理是 Agent 开发的基础。本章从工程实践角度讲解 Token 化、Transformer 架构、Attention 机制和采样策略的核心概念。掌握这些知识后，你能够更准确地估算成本、选择合适的模型参数，并理解为什么 LLM 会有上下文窗口限制。

19.1 Token 化原理

什么是 Token

Token 是 LLM 处理的最小文本单位，介于单词和字符之间。理解 Token 划分对成本估算和 Prompt 优化至关重要。

英文 Token 划分： - 常见单词是一个 Token（如 "cat"、"run"） - 复杂单词会被拆分（如 "playing" → "play" + "ing"） - 词缀通常独立（如 "un-believ-able" 可能是 3-4 个 Token）

中文 Token 划分： - 单个汉字通常是一个 Token（如 "漫"、"剧"） - 常见词组可能合并（如 "漫剧" 可能是一个 Token） - 生僻词会被拆分（如 "轩辕墨" → "轩辕" + "墨"）

经验换算： - 英文：100 字符 ≈ 75 Token（约 0.75 倍） - 中文：1 汉字 ≈ 1-2 Token（取决于词频）

为什么 Token 划分影响理解：生僻角色名被拆分后，LLM 可能无法识别为完整实体。例如「轩辕墨」被拆分为「轩辕」和「墨」，LLM 可能误以为是两个独立概念。在漫剧设定管理中，生僻名称首次出现时应明确标注「这是一个角色名」。

Token 化方法

Token 化由独立的 Tokenizer 组件完成，不同模型使用不同算法。了解这些方法有助于理解为什么同一文本在不同模型上 Token 数量不同。

BPE（Byte Pair Encoding）： - **原理**：从字符级别开始，反复合并最常见的字符对，形成词表 - **过程**：初始词表是所有字符 → 统计最频繁字符对 → 合并为新 Token → 重复直到词表达到目标大小（如 30K-100K） - **特点**：高频词是完整 Token，低频词被拆分 - **使用模型**：GPT 系列、Llama 系列

WordPiece： - **原理**：类似 BPE，但合并时考虑词边界（用 ## 标记词内位置） - **特点**：能更好保留词义信息 - **使用模型**：BERT、早期 Google 模型

Unigram（SentencePiece）： - **原理**：从大词表开始，逐步删除低频词，直到达到目标大小 - **特点**：支持多语言混合，不依赖预定义词表 - **使用模型**：T5、部分多语言模型

为什么需要 Tokenizer：LLM 只能处理数字序列。Tokenizer 将任意文本映射到固定词表（如 0-99999 的整数），LLM 才能进行数学运算。不同模型的词表不同，因此同一文本在不同模型上的 Token 数可能差异很大。

工程影响： - 跨模型迁移时，重新估算 Token 成本 - 生僻词、专业术语可能增加 Token 数 - 特殊符号（如 emoji、代码符号）通常各占一个 Token

Token 与成本计算

LLM API 按 Token 计费，输入和输出分别计算。准确估算 Token 用量是成本控制的关键。

计费方式： - **输入 Token**：Prompt + 上下文 + 工具描述 + Few-shot 示例 - **输出 Token**：LLM 生成的全部内容 - **价格差异**：输出 Token 通常比输入贵 2 倍（因为生成需要更多计算）

成本计算公式： `单次调用成本 = (输入 Token × 输入单价) + (输出 Token × 输出单价)`

示例（以 GPT-4 为例）： - 输入单价：$0.03 / 1K Token - 输出单价：$0.06 / 1K Token - 漫剧大纲生成：输入 2000 Token + 输出 1500 Token - 成本 = (2000 × 0.03/1000) + (1500 × 0.06/1000) = $0.06 + $0.09 = $0.15

隐藏成本： - **重试**：失败重试增加 1.2-1.5 倍成本 - **工具调用**：每次 Function Calling 都是一次独立 API 调用 - **嵌入计算**：向量嵌入生成也消耗 Token（部分模型单独计费）

优化策略： 1. **压缩 Prompt**：移除冗余描述，用简洁语言表达 2. **限制输出长度**：设置 `max_tokens` 参数，避免生成长度失控 3. **缓存重复查询**：相同输入直接返回缓存结果 4. **批量处理**：多个相似任务合并为一次调用

漫剧生成案例：分析发现 Prompt 占输入 Token 的 60%。通过移除冗余描述、用缩写替代长术语，单次生成 Token 从 5000 降至 3000，成本降低 40%，质量无明显下降。

19.2 Transformer 架构简介

Encoder-Decoder 结构

Transformer（变换器，Google 2017 年提出的深度学习架构）是现代 LLM 的基础架构，理解其结构有助于选择合适的模型类型。

Encoder（编码器）： - **功能**：编码输入序列，提取语义表示 - **特点**：能看到完整输入序列，适合理解任务 - **适用场景**：文本分类、命名实体识别、语义检索

Decoder（解码器）： - **功能**：基于编码生成输出序列 - **特点**：只能看到当前位置之前的内容（因果掩码），适合生成任务 - **适用场景**：文本生成、对话、代码生成

三种架构模式：

架构类型	代表模型	适用场景
Encoder-only	BERT	理解任务（分类、检索）
Decoder-only	GPT 系列	生成任务（对话、创作）
Encoder-Decoder	T5、BART	转换任务（翻译、摘要）

为什么 Agent 用 Decoder-only：Agent 的核心任务是生成（回复、规划、工具调用）。Decoder-only 模型逐个 Token 生成，天然适合对话和连续文本生成。

工程影响：- 理解任务（如设定检索）可用 Encoder-only 模型生成嵌入 - 生成任务（如大纲、正文）必须用 Decoder-only 模型 - 不要试图用 BERT 做文本生成（它不是为生成设计的）

自注意力机制概念

自注意力（Self-Attention）是 Transformer 的核心创新，解决了 RNN/LSTM 的长距离依赖问题。

RNN/LSTM 的局限：- **顺序处理**：必须按顺序处理每个 Token，无法并行 - **长距离依赖弱**：序列开头和结尾的信息难以传递（梯度消失） - **速度慢**：1000 Token 需要 1000 步，无法加速

自注意力的解决方案：- **核心思想**：序列中每个 Token 直接关注所有其他 Token，无论距离 - **计算方式**：每个 Token 与其他 Token 计算关联度，加权聚合信息 - **优势 1**：并行计算（所有 Token 同时处理），速度大幅提升 - **优势 2**：长距离依赖（开头和结尾直接关联），信息传递无损耗

为什么需要自注意力：漫剧大纲生成时，开头设定的伏笔需要与结尾呼应。RNN 可能「忘记」开头内容，但自注意力让模型能同时关注开头和结尾，确保剧情连贯。

局限性：- **上下文窗口限制**：虽然理论上能关注任意长度，但实际受计算复杂度限制 - **不是真正记忆**：自注意力只在单次推理内有效，跨会话需要外部记忆（向量数据库）

位置编码作用

位置编码（Positional Encoding）让 Transformer 理解 Token 的顺序信息。

问题：自注意力是置换不变的（打乱 Token 顺序，计算结果不变），但语言需要顺序信息。

解决方案：给每个位置添加位置编码，让模型知道每个 Token 的位置。

实现方式： - **正弦余弦函数**（原始 Transformer）：用数学函数生成位置向量，能外推到未见过的长度 - **学习的位置嵌入**（GPT 系列）：像词嵌入一样训练位置向量，效果更好但无法外推

为什么需要位置编码：漫剧剧情生成中，「主角先获得能力，再遇到挑战，最后战胜敌人」的顺序至关重要。没有位置编码，模型无法理解时间顺序，可能生成混乱的剧情。

工程影响： - 位置编码限制了最大上下文窗口（如 GPT-4 训练时最大 8K，无法直接处理 16K） - 外推技术（如 RoPE、ALiBi）能部分突破限制，但可能影响质量 - 长文本处理仍需分块或摘要

19.3 Attention 机制

Q、K、V 的概念与作用

Q（Query）、K（Key）、V（Value）是 Attention 机制的三个核心组件。

定义： - **Query（查询向量）**：当前 Token 的「查询」，表示「我想找什么信息」 - **Key（关键向量）**：所有 Token 的「标签」，表示「我提供什么信息」 - **Value（值向量）**：所有 Token 的「内容」，表示「我的实际信息」

计算过程： 1. 当前 Token 的 Q 与所有 Token 的 K 计算相似度（点积） 2. 相似度经过 Softmax（归一化函数，将数值转换为概率分布）归一化，得到权重（0-1 之间） 3. 用权重加权聚合所有 V，得到当前 Token 的新表示

为什么 QKV 不一样： - **解耦查询、匹配、内容**：让模型更灵活地学习不同关系 - **类比数据库**：Q 是搜索关键词，K 是索引，V 是实际数据 - **如果 QKV 相同**：模型表达能力受限，无法区分不同角色

漫剧设定一致性检查案例： - Q：「角色能力」查询 - K：各设定片段的主题标签（如「世界观」「角色 A」「角色 B」） - V：设定内容（如「角色 A 能飞行，最高速度 100km/h」） - 结果：找到与「角色能力」相关的设定片段，进行一致性校验

常见误区：认为 QKV 是固定的。实际每个注意力头有独立的 QKV 矩阵，通过训练学习不同的关注模式。

Multi-head Attention 简介

Multi-head Attention（多头注意力）并行运行多个注意力头，每个头关注不同方面。

核心思想：单一注意力只能捕捉一种关系，多头像多个视角同时观察。

工作机制： 1. 输入分别通过多个独立的 QKV 投影（如 96 个头就有 96 组 QKV 矩阵） 2. 每个头独立计算 Attention 3. 所有头的输出拼接，通过线性层整合

为什么需要多头： - **多视角理解**：一个头关注语法关系，一个头关注语义关系，一个头关注长距离依赖 - **增强表达能力**：多头能同时捕捉多种模式 - **头数选择**：GPT-3 有 96 头，GPT-4 推测 100+ 头，越多表达能力越强

漫剧大纲生成案例： - 头 1：关注剧情逻辑（因果关系） - 头 2：关注角色关系（谁和谁互动） - 头 3：关注节奏（高潮、低谷分布） - 头 4：关注伏笔（前后呼应） - 整合：生成结构完整、逻辑连贯的大纲

常见误区：认为头数越多越好。实际增加头数会提升计算成本，且可能过拟合。中小模型通常 8-16 头就够了。

Attention 复杂度分析

Attention 的计算复杂度是 $O(n^2)$，这是上下文窗口受限的根本原因。

为什么是 $O(n^2)$： - n 个 Token，每个 Token 的 Q 需要与所有 n 个 K 计算相似度 - 总计算量 = $n \times n = n^2$ 次点积运算 - 序列长度翻倍，计算量变为 4 倍

影响： - **计算慢**：长序列推理时间大幅增加 - **显存占用大**：需要存储 n^2 的注意力矩阵 - **上下文窗口限制**：GPT-4 通常 8K-128K，无法无限扩展

优化方向： - **稀疏 Attention**：只关注部分位置（如相邻 Token、关键 Token），降低到 $O(n \log n)$ 或 $O(n)$ - **线性 Attention**：用近似计算替代精确 Attention，降低到 $O(n)$ - **分块处理**：长文本分块，每块独立计算 Attention

工程影响：漫剧长文本生成受 Attention 复杂度限制，无法一次性生成 10 万字。解决方案是分章节生成，每章控制在上下文窗口内（如 8K Token）。

19.4 温度与采样策略

温度（Temperature）的作用

温度（Temperature）控制输出的确定性和多样性，是调节 LLM 行为最重要的参数之一。

定义：温度缩放 logits（未归一化的概率），影响概率分布的平滑程度。

温度的影响： | 温度值 | 概率分布 | 输出特点 | 适用场景 | |-------|---------|---------|---------| | T < 1（低温） | 更尖锐，高概率词更高 | 确定、保守、重复风险 | 事实问答、代码生成 | | T = 1 | 原始分布 | 平衡 | 通用对话 | | T > 1（高温） | 更平滑，低概率词也有机会 | 多样、创意、混乱风险 | 创意生成、头脑风暴 | | T = 0 | 贪婪采样，总是选概率最高 | 最确定、最重复 | 数据提取、格式化输出 |

为什么温度有效：低温让高概率词「赢家通吃」，模型倾向于选择最安全的答案；高温让低概率词也有机会，模型更愿意尝试新颖组合。

漫剧生成参数建议： - **创意收集**：T = 0.8-0.9（鼓励创意，允许一定跳跃） - **设定文档**：T = 0.3-0.5（确保准确性，减少幻觉） - **章节正文**：T = 0.6-0.7（平衡创意和可控） - **角色对话**：T = 0.7-0.8（让对话自然，避免机械感）

常见误区：认为温度越高创意越好。实际 T > 1.0 时输出可能变得混乱，失去可读性。创意和可控需要平衡。

Top-p、Top-k 采样

Top-p 和 Top-k 是两种常用的采样策略，通常与温度组合使用。

Top-k 采样： - **定义**：从概率最高的 k 个词中随机采样 - **参数**：k 是固定值（如 k = 50） - **优点**：简单直观，排除低概率词 - **缺点**：k 固定，可能在某些步太多或太少

Top-p 采样（Nucleus Sampling）： - **定义**：从累积概率达到 p 的最小词集中采样 - **参数**：p 是阈值（如 p = 0.9，表示覆盖 90% 概率） - **优点**：动态调整候选词数量，概率集中时词少，分散时词多 - **缺点**：参数理解稍复杂

为什么需要 Top-p：固定 k 在某些情况下不合适。例如概率集中时（明显正确答案），k=50 可能包含太多低质量候选；概率分散时（多个合理答案），k=50 可能排除合理选项。Top-p 自动调整，更灵活。

组合使用：通常同时设置 Top-k 和 Top-p，取两者交集。例如 Top-p=0.9、Top-k=40，表示从累积概率 90% 的词中选前 40 个。

漫剧角色对话参数建议： - Top-p = 0.9（覆盖 90% 概率，排除极端低概率词） - Top-k = 40（最多 40 个候选，避免太分散） - T = 0.7-0.8（平衡自然度和可控性）

常见误区：认为 Top-p 和 Top-k 只能选一个。实际可以组合使用，提供更精细的控制。

创造性 vs 确定性任务

根据任务类型选择合适的温度和采样参数，是提升输出质量的关键。

任务分类与参数建议:

| 任务类型 | 温度 (T) | Top-p | Top-k | 说明 | |---------|---------|-------|-------|------| | **创意生成** | 0.7-0.9 | 0.9-0.95 | 40-60 | 故事、诗歌、头脑风暴 | | **对话聊天** | 0.5-0.7 | 0.8-0.9 | 30-50 | 平衡自然度和相关性 | | **事实问答** | 0.1-0.3 | 0.1-0.5 | 10-20 | 确保准确性，减少幻觉 | | **代码生成** | 0.1-0.3 | 0.1-0.5 | 10-20 | 语法正确优先 | | **数据提取** | 0 | - | - | 贪婪采样，确保格式 | | **翻译** | 0.3-0.5 | 0.5-0.7 | 20-30 | 准确传达原意 |

调试策略： 1. **先低温确保正确**：T=0.3 测试基本功能 2. **逐步提高温度**：每次 +0.1，观察输出变化 3. **找到平衡点**：质量开始下降时的前一个温度 4. **记录最佳参数**：不同任务类型保存不同配置

漫剧生成全流程参数配置： `创意沟通 → T=0.8, Top-p=0.9, Top-k=40` `设定生成 → T=0.3, Top-p=0.5, Top-k=20` `大纲规划 → T=0.6, Top-p=0.85, Top-k=30` `章节正文 → T=0.6, Top-p=0.9, Top-k=40` `角色对话 → T=0.7, Top-p=0.9, Top-k=40`

常见误区：所有任务用同一套参数。实际应根据任务类型调整，创造性任务用高温高 Top-p，确定性任务用低温低 Top-p。

19.5 简单举例

案例设计

- **案例名称**：Token 优化在漫剧生成中的应用
- **涉及知识点**：Token 化原理、Transformer 架构、Attention 机制、温度与采样策略
- **案例目标**：帮助理解如何通过 Token 优化降低漫剧生成成本
- **案例内容要点**:
- 场景描述：漫剧章节生成成本过高，单次生成平均消耗 5000 Token，按 GPT-4 价格计算每章$0.3，整部漫剧$6 超出预算
- 技术应用：用 tiktoken 分析 Token 构成，Prompt 压缩移除冗余描述减少 35% 输入 Token，设置 max_tokens=3000 限制输出长度，温度从 0.8 降至 0.6 减少重试
- 效果说明：单次生成 Token 从 5000 降至 3000，成本降低 40%（$0.3→$0.18），整部漫剧成本从$6 降至$3.6，质量无明显下降
- **注意事项**：不展开 tiktoken 的具体使用细节（见第 21 章）

知识来源: - Transformer 论文：Attention Is All You Need (Vaswani et al., 2017) - https://arxiv.org/abs/1706.03762 - GPT-3 论文：Language Models are Few-Shot Learners (Brown et al., 2020) - https://arxiv.org/abs/2005.14165 - OpenAI API 文

档：Temperature and Sampling Parameters - https://platform.openai.com/docs/api-reference/chat/create

---

修改记录: - v2.0 (2026-03-23): 润色版 — 句子简化、删除重复、优化结构、统一语气 - v1.1 (2026-03-22): 根据编辑统筹意见修改 — 规范知识来源格式 - v1.0 (2026-03-22): 初稿完成